HCRTOS SDK pwm userguide

1. Table of

2. Documentation

Version Number	Date	Created/Revised Person	Create/Revision History
1.0	2023.04.10	Qiu Haojia	Added Explanatory Document
2.0	2023.04.23	Qiu Haojia	Supports setting duty cycle and period during initialization
3.0	2023.04.25	Qiu Haojia	Cancel setting duty cycle and period during initialization, and avoid overlap at different stages Sub-initialization
4.0	2023.08.14	Qiu Haojia	Added Polarity Setting
5.0	2023.09.09	Qiu Haojia	Added Configurable PWM Range Selection

3. Overview

3.1 Purpose of writing

Guidance and introduction to the use and development of PWM

3.2 Target readers

Software development engineers and technical support engineers.

4. Module

- The RTOS PWM module distinguishes between the 15xx series and 16xx series chips; the 15xx series has 3 PWM channels, the 16xx series has 6 PWM channels.
- In the current driver, the maximum PWM clock is 27M, and the maximum period at 27M clock is 37ns.
- Supports setting duty cycle and period during driver initialization.

4.1 Device Tree Configuration

Taking pwm0 as an example:

```
pwm@0 {
            pinmux-active = <PINPAD_L23 1>;
            devpath = "/dev/pwm0";
            polarity = <1>;
            status = "okay";
};
```

pinmux-active: Configure the pin PINPAD_L23 to PWM mode. It does not mean that all PWM values are set to 1. Please refer to the specific configuration for the pins used.

references: components/kernel/source/include/uapi/hcuapi/pinmux/hc15xx_pinmux.h and components/kernel/source/include/uapi/hcuapi/pinmux/hc16xx_pinmux.h.

devpath: Path of the generated PWM node.

polarity: Polarity of the channel after initialization. 0: low level; 1: high level.

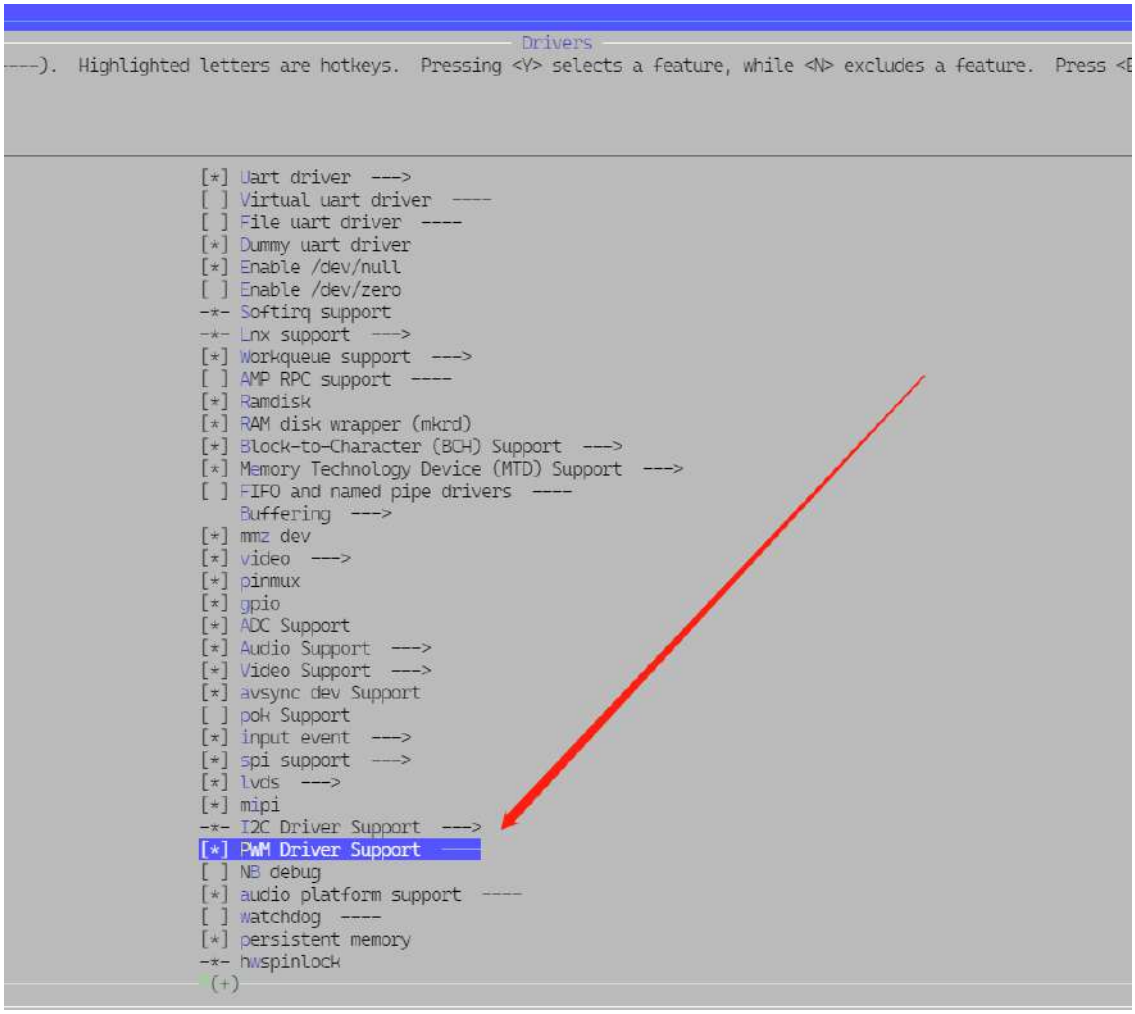
status: 'okay' means enabled; 'disabled' means not enabled.

PWM duty cycle and frequency are set through code; please refer to the sample code.

4.2 Menuconfig configuration

In the SDK root directory, run make menuconfig and select PWM under the following path.

```
Location:
  -> Components
    -> kernel (BR2_PACKAGE_KERNEL [=y])
      -> Drivers
```



4.2.1 PWM frequency range selection

To meet PWM having 0-100% and each adjustable level at 1%, you can set values within the PWM range, but you can also set frequencies beyond the maximum of the range; however, the adjustable level becomes uncertain and varies with the configured frequency.

When only one channel is used, CONFIG_RANGE_AUTO_SET can be configured, and the division factor will be calculated automatically. However, beyond the range of this division factor, the adjustable level is uncertain.

After configuration is complete, enter make kernel-rebuild all to compile and burn, and enter the following commands in the serial console to view the pwm node.

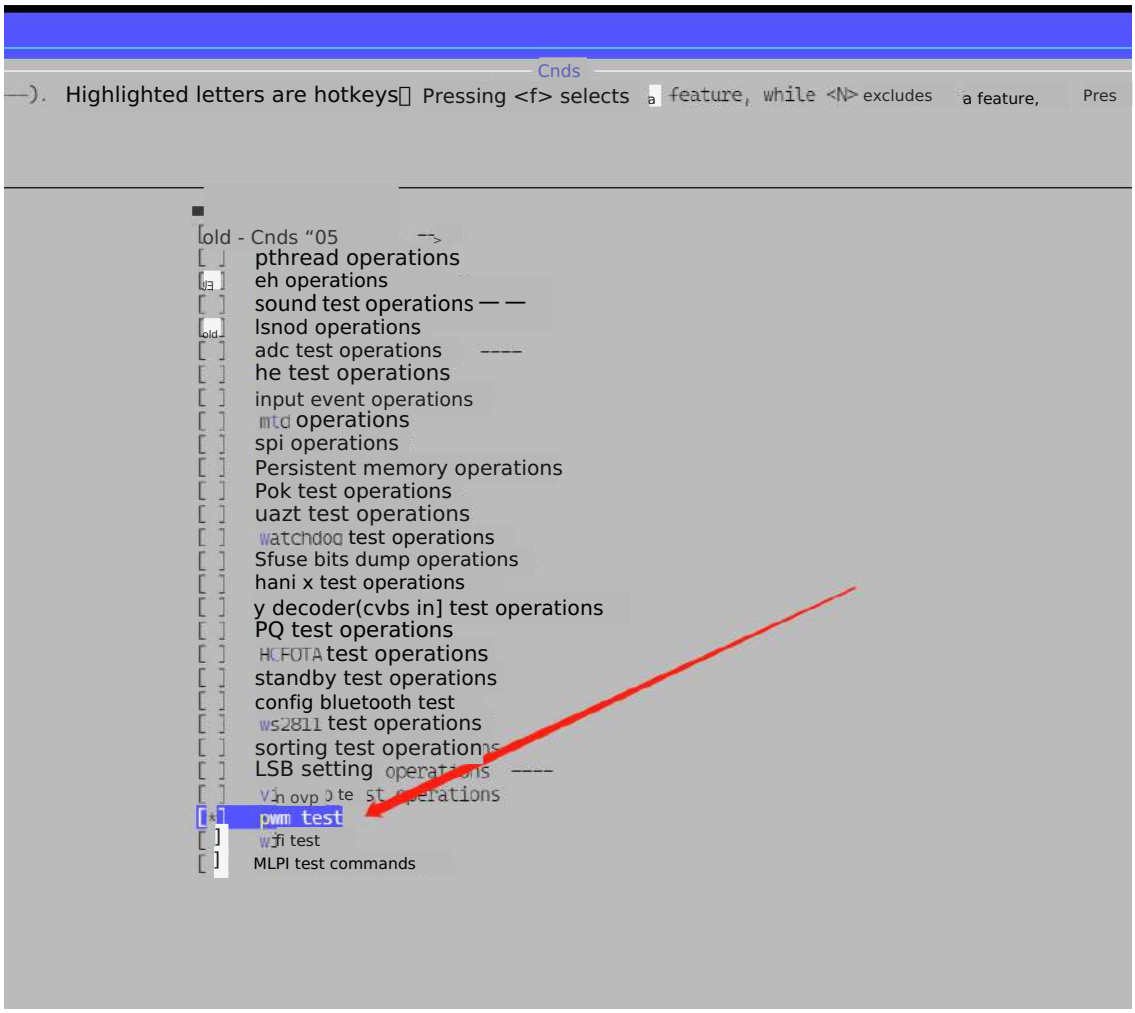
```
hc1512aGdbB266#
hc1512aGdbB200# nsh
hc1512a@dbB266 (nsh)# 1s
/:
 dev/
hc1512a@dbB200 (nsh)# cd    dev
□□ hc1512aGdbB269 (nsh)# 1s :

 auddec
 audsink
 avsync0
 avsync1
 bus/
 dis
 fb0
 ge
 input/
mmz
mtdblock6
mtdblock1
mtdblock2
mtdblock3
mtdblock4
 null
nersJ s+pntmem
Pwm6
s T_prodect
 sndC0i2so
 uart1
uart_dummy
viddec
vidsink
hc1512a@dbB200 (nsh)# phase
```

4.3 test command

In the SDK root directory, enter make menuconfig and select the pwm_test command according to the path below.

```
Location:
  -> Components
    -> Cmds (BR2_PACKAGE_CMDS [=y])
```



After configuration is complete, enter make cmds-rebuild all to compile and burn, and input the following commands in the serial console to enable test commands.

```
hc1512a@dbB200#
hc1512a@dbB200#
hc1512a@dbB200# pwm_test -h
Usage: pwm_test
  -s --start                    start pwm device id
  -p --period_ns          set period_ns
  -d --duty_ns            set duty_ns
  -P --ploarity           set ploarity
  -S --stop               stop pwm device id

Invalid command "pwm_test"
hc1512a@dbB200# pwm_test -s0 -p1000000 -d400000 -P0
start /dev/pwm0 test
hc1512a@dbB200#
```

5. Module interface

5.1 Interface functions

This module provides ioctl function interfaces for the application layer.

- Set PWM parameters, including duty cycle, period, and polarity. Note that the units for duty cycle and period are ns.

```
info.polarity = polarity;
info.period_ns = period_ns;
info.duty_ns = duty_ns;
ioctl(fd, PWMIOC_SETCHARACTERISTICS,              (unsigned long)&info);
```

- Get PWM parameters, including duty cycle, period, and polarity. Note that the units for duty cycle and period are ns.

```
info.polarity = polarity;
info.period_ns = period_ns;
info.duty_ns = duty_ns;
ioctl(fd, PWMIOC_GETCHARACTERISTICS,              (unsigned long)&info);
```

- Start and stop PWM.

```
ioctl(fd, PWMIOC_START);
ioctl(fd, PWMIOC_STOP);
```

6. Module test cases and sample code

Code is stored at: components/cmds/source/pwm/pwm_test.c. Note that the units for duty cycle and period are ns.

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <poll.h>
#include <signal.h>
#include <stdlib.h>
#include <string.h>
#include <getopt.h>
#include <sys/ioctl.h>
#include <hcuapi/pwm.h>
#include <kernel/lib/console.h>

static void print_usage(const char *prog)
{
        printf("Usage: %s \n", prog);
        puts("       -s --start                    start pwm device id\n"
          " -p --period_ns                  set period_ns\n"
          " -d --duty_ns                    set duty_ns\n"
          " -P --ploarity                   set ploarity\n"
          " -S --stop                   stop pwm device id\n");
}

static int pwm_test(int argc, char *argv[])
{
```

```
25      int fd;
26      int id = 0;
27      bool stop_t = false;
28      char path[64] = { 0 };
29      struct pwm_info_s info = { 0 };
30      uint32_t period_ns, duty_ns, polarity;
31      period_ns = 1000000;
32      duty_ns = 500000;
33      polarity = 0;
34
35      opterr = 0;
36      optind = 0;
37
38      while (1) {
39          static const struct option lopts[] = {
40              { "start",                  1, 0, 's' },
41              { "period_ns",              1, 0, 'p' },
42              { "duty_ns",                1, 0, 'd' },
43              { "stop",                   1, 0, 'S' },
44              { "polarity",               1, 0, 'P' },
45              { "NULL",               0, 0, 0 },
46          };
47
48          int c;
49
50          c = getopt_long(argc, argv, "s:p:d:S:P:", lopts, NULL);
51
52          if (c == -1)        {
53              break;
54          }
55
56          switch(c)       {
57          case 's':
58              id = atoi(optarg);
59              break;
60          case 'p':
61              period_ns = atoi(optarg);
62              break;
63          case 'P':
64              polarity = atoi(optarg);
65              break;
66          case 'd':
67              duty_ns = atoi(optarg);
68              break;
69          case 'S':
70              stop_t = true;
71              id = atoi(optarg);
72              break;
73          default:
74              print_usage(argv[0]);
75              return -1;
76          }
77      }
78
79      sprintf(path, "/dev/pwm%d", id);
80      fd = open(path, O_RDWR);
81      if (fd < 0)         {
82          printf("%s       open fail\n", path);
```

```
            return 0;
        }

        if (stop_t == true)             {
            printf("stop        %s test\n", path);
            ioctl(fd, PWMIOC_STOP);
            close(fd);
            return 0;
        }

        info.polarity = polarity;
        info.period_ns = period_ns;
        info.duty_ns = duty_ns;
        ioctl(fd, PWMIOC_SETCHARACTERISTICS,            (unsigned long)&info);

        ioctl(fd, PWMIOC_START);
        close(fd);
        printf("start         %s test\n", path);

        return 0;
}

CONSOLE_CMD(pwm_test, NULL, pwm_test, CONSOLE_CMD_MODE_SELF, "pwm test
operations")
```

7. Module debugging

Can use an oscilloscope or logic analyzer to capture waveforms for viewing.

8. Frequently Asked

None for now.